

# Reading the AgentCom Starter Code

Course notes

August 27, 2026

## Contents

|                                                                         |           |
|-------------------------------------------------------------------------|-----------|
| <b>1 Overview</b>                                                       | <b>2</b>  |
| <b>2 <code>agents.py</code>: what an agent is</b>                       | <b>2</b>  |
| <b>3 <code>budget.py</code>: the guardrail</b>                          | <b>2</b>  |
| 3.1 Constructor . . . . .                                               | 2         |
| 3.2 The clock . . . . .                                                 | 3         |
| 3.3 Recording cost . . . . .                                            | 3         |
| 3.4 Stopping . . . . .                                                  | 4         |
| <b>4 <code>llm_client.py</code>: talking to a model</b>                 | <b>4</b>  |
| 4.1 The response type . . . . .                                         | 4         |
| 4.2 <code>OllamaClient</code> . . . . .                                 | 5         |
| 4.3 <code>MockClient</code> . . . . .                                   | 6         |
| <b>5 <code>ping_pong.py</code>: the Week 1 loop</b>                     | <b>6</b>  |
| <b>6 <code>engine.py</code>: the Week 2 target</b>                      | <b>7</b>  |
| 6.1 Entry . . . . .                                                     | 7         |
| 6.2 <code>view_for</code> : to be written . . . . .                     | 7         |
| 6.3 <code>DialogueEngine</code> . . . . .                               | 8         |
| 6.4 <code>run</code> : to be written . . . . .                          | 8         |
| 6.5 <code>save</code> : done . . . . .                                  | 8         |
| <b>7 <code>run.py</code>: the entry point</b>                           | <b>9</b>  |
| <b>8 <code>debate.yaml</code>: the config</b>                           | <b>9</b>  |
| <b>9 <code>test_engine.py</code>: the self-check</b>                    | <b>10</b> |
| 9.1 Setup . . . . .                                                     | 10        |
| 9.2 Checking <code>view_for</code> . . . . .                            | 11        |
| 9.3 Checking <code>run</code> . . . . .                                 | 11        |
| 9.4 Checking <code>save</code> . . . . .                                | 12        |
| <b>10 Smaller pieces not covered above</b>                              | <b>12</b> |
| 10.1 <code>README.md</code> and <code>requirements.txt</code> . . . . . | 12        |
| 10.2 The <code>__main__</code> blocks . . . . .                         | 12        |
| 10.3 <code>MockClient.DEFAULT_REPLIES</code> . . . . .                  | 12        |
| 10.4 <code>Budget.summary</code> . . . . .                              | 13        |
| 10.5 The <code>totals</code> block in <code>save</code> . . . . .       | 13        |

## 1 Overview

The whole project is built around one idea: two language-model “agents” take turns talking to each other, and a guardrail stops them before they run away with your time or your tokens. Every file plays one small role in that.

| File / object                                          | Job                              |
|--------------------------------------------------------|----------------------------------|
| Budget in <code>budget.py</code>                       | decide when to stop              |
| OllamaClient, MockClient in <code>llm_client.py</code> | talk to a model                  |
| Agent in <code>agents.py</code>                        | describe who is speaking         |
| view_for in <code>engine.py</code>                     | build what the model sees        |
| DialogueEngine.run in <code>engine.py</code>           | tie the above together in a loop |
| DialogueEngine.save in <code>engine.py</code>          | write the result to JSON         |

`ping_pong.py` (Week 1) does all of this in one rough script. `engine.py` (Week 2) does the same thing properly, with real chat roles.

## 2 `agents.py`: what an agent is

```
from dataclasses import dataclass

@dataclass
class Agent:
    name: str
    system_prompt: str
    model: str = "llama3.2:3b"
    temperature: float = 0.7
```

`@dataclass` is a decorator that writes the boring parts of a class for you. Given the four field declarations, Python generates an `__init__` that takes them as arguments, a readable `__repr__`, and an equality test. The fields with an `=` have defaults, so `Agent("Pro", "Argue in favour.")` is a complete agent.

Notice that there are no methods. An agent is nothing but a name, a system prompt (its persona), a model tag, and a temperature (how random the model’s sampling is). The docstring says this plainly: an agent has no behaviour of its own. The engine renders the conversation from the agent’s point of view, sends that to the model, and the model’s reply is what the agent “does”.

## 3 `budget.py`: the guardrail

This file is complete and you are told not to change it. It is the piece that makes the rest of the project safe to run.

### 3.1 Constructor

```
def __init__(self, max_turns=10, max_tokens=4000, max_seconds=120):
    self.max_turns = max_turns
    self.max_tokens = max_tokens
```

```

self.max_seconds = max_seconds
self.turns = 0
self.tokens = 0
self._start = None
self.stop_reason = None

```

Three limits, three things that track progress against them. `turns` and `tokens` start at zero and are incremented later. Time is handled differently: `_start` is `None` until the clock is started, and the elapsed time is computed whenever it is needed rather than stored. `stop_reason` stays `None` until some limit fires.

### 3.2 The clock

```

def start(self):
    self._start = time.monotonic()
    return self

@property
def elapsed(self):
    return 0.0 if self._start is None else time.monotonic() - self._start

```

`start` records the current time. It uses `time.monotonic()` rather than `time.time()` because a monotonic clock can only move forward; the wall clock can jump if the operating system adjusts it, and a negative elapsed time would be nonsense. Returning `self` is a small convenience that lets you write `Budget(...).start()` on one line.

`elapsed` is a single line, but it packs in two things worth pausing on.

The body is a conditional expression, which is Python's one-line form of if/else. Spelled out, it reads:

```

if self._start is None:
    return 0.0                                # clock never started
else:
    return time.monotonic() - self._start      # now minus then

```

The other thing is the `@property` line above the `def`. Without it, `elapsed` would be an ordinary method and you would call it with parentheses: `b.elapsed()`. With it, Python lets you access it as if it were a plain attribute, `b.elapsed`, and quietly runs the function each time. The value is recomputed on every access, which is exactly what you want for something that changes continuously. From the outside, `b.turns` (a stored number) and `b.elapsed` (a computed one) look the same, and that is the point of the decorator.

### 3.3 Recording cost

```

def record(self, turns=0, tokens=0):
    self.turns += turns
    self.tokens += tokens

```

Called once after every model reply. `+=` adds to the running total, so `self.turns += turns` is shorthand for `self.turns = self.turns + turns`. Both parameters default to zero so you can pass either one alone. In practice the call is always

```

budget.record(turns=1, tokens=reply.tokens)

```

so after three replies costing 50, 60 and 70 tokens you have `turns == 3` and `tokens == 180`.

Compare this with `elapsed`. Turns and tokens have to be recorded because nothing else knows about them; time does not, because the clock is always there to be read. That asymmetry is why one is a property and the other two are plain counters.

### 3.4 Stopping

```
def stop(self, reason):
    if self.stop_reason is None:
        self.stop_reason = reason
```

Lets the caller force a stop for a reason of its own, for instance "goal\_reached" when the agents agree. The `if` means the first reason recorded is the one that sticks; a later call cannot overwrite it, which keeps the transcript honest about why the run ended.

```
def exhausted(self):
    if self._start is None:
        self.start()
    if self.stop_reason is not None:
        return True
    if self.turns >= self.max_turns:
        self.stop("max_turns")
        return True
    if self.tokens >= self.max_tokens:
        self.stop("max_tokens")
        return True
    if self.elapsed >= self.max_seconds:
        self.stop("max_seconds")
        return True
    return False
```

This is the method the main loop asks every iteration. The first call starts the clock, so you never have to remember to. After that it checks, in order: has something already stopped us, then the three limits. Whichever trips first records its name and returns `True`. The loop is always written while not `budget.exhausted()` :, so a `True` here ends it.

`summary()` just packages the counters and limits into a dictionary so `save` can dump them to JSON.

## 4 `llm_client.py`: talking to a model

### 4.1 The response type

```
@dataclass
class ChatResponse:
    text: str
    prompt_tokens: int
    completion_tokens: int
    seconds: float

    @property
    def tokens(self):
        return self.prompt_tokens + self.completion_tokens
```

Every model call returns one of these: the reply text, how many tokens were sent in (the prompt), how many came back (the completion), and how long the call took. `tokens` is the

sum, and it is a property for the same reason `elapsed` was: it is derived from the other two fields, so there is no point storing it separately.

## 4.2 OllamaClient

```
def __init__(self, host="http://localhost:11434"):
    self.host = host
```

Ollama runs a small web server on your machine; 11434 is its default port.

```
def chat(self, model, messages, temperature=0.7):
    t0 = time.monotonic()
    try:
        resp = requests.post(
            f"{self.host}/api/chat",
            json={
                "model": model,
                "messages": messages,
                "stream": False,
                "options": {"temperature": temperature},
            },
            timeout=300,
        )
    except requests.ConnectionError:
        raise RuntimeError("Cannot connect to Ollama ...") from None
```

The `requests.post` call is where the conversation is actually sent to the model. Because Ollama speaks HTTP, this is no different from calling any web API. The URL is built from the host and the path `/api/chat`, so by default it is `http://localhost:11434/api/chat`. The `json=` argument is the request body; `requests` turns the dictionary into JSON and sets the content-type header for you. Inside it, `"model"` names the model to run, `"messages"` is the list of role/content pairs that make up the conversation, and `"options"` carries the temperature. `"stream": False` is worth a note: Ollama's default is to stream the answer back a few tokens at a time, which is nice for a chat window and annoying for a program. Setting it to `False` means "wait and give me the finished reply".

`timeout=300` is a small guardrail of its own. Without it, a stuck Ollama process would leave your program hanging forever. Five minutes is generous because small models on a laptop can be slow.

The `except` clause catches the case where nothing is listening on that port and replaces the raw network error with a message that tells you what to do. `from None` suppresses the original traceback so the friendly message is all you see.

```
if resp.status_code == 404:
    raise RuntimeError("Model not found ...")
resp.raise_for_status()
data = resp.json()
return ChatResponse(
    text=data["message"]["content"].strip(),
    prompt_tokens=data.get("prompt_eval_count", 0),
    completion_tokens=data.get("eval_count", 0),
    seconds=time.monotonic() - t0,
)
```

A 404 from Ollama means the model tag has not been pulled. `raise_for_status` turns any other HTTP error into an exception. Then the JSON body is parsed and the parts we care about are copied into a `ChatResponse`. The token counts come straight from Ollama, so they

are exact.

It is worth noticing that this method is the only place in the entire project that knows anything about Ollama. Everything else calls `client.chat(...)` and does not care what is behind it. That is what makes the mock client possible.

### 4.3 MockClient

```
def __init__(self, replies=None):
    self._replies = replies or self.DEFAULT_REPLIES
    self._i = 0

def chat(self, model, messages, temperature=0.7):
    text = self._replies[self._i % len(self._replies)]
    self._i += 1
    prompt_tokens = sum(len(m["content"].split()) for m in messages)
    return ChatResponse(
        text=text,
        prompt_tokens=prompt_tokens,
        completion_tokens=max(1, len(text.split())),
        seconds=0.01,
    )
```

A fake model. It holds a short list of canned replies and a counter, and each call hands out the next reply, wrapping back to the start thanks to the modulo. Token counts are approximated by counting words, which is crude but enough to make the Budget behave realistically. The method has exactly the same signature as the real one, so the engine cannot tell which it is talking to.

```
def make_client(mock=False, host="http://localhost:11434"):
    return MockClient() if mock else OllamaClient(host=host)
```

A one-line factory. The `-mock` flag on the command line ends up here.

## 5 ping\_pong.py: the Week 1 loop

This script works as delivered. It exists to show the shape of a two-agent loop before you build the real thing.

```
TOPIC = "Cities should ban private cars from their centres."
AGENT_A = Agent("Pro", "You argue IN FAVOUR of the topic. Be brief: 2
    sentences max.")
AGENT_B = Agent("Con", "You argue AGAINST the topic. Be brief: 2
    sentences max.")

def render(transcript):
    if not transcript:
        return "(You speak first.)"
    return "\n".join(f"{name}: {text}" for name, text in transcript)
```

Two personas, and a helper that flattens the transcript (a list of `(name, text)` pairs) into a block of text with one line per message.

```
def main(mock, turns):
    client = make_client(mock=mock)
    agents = [AGENT_A, AGENT_B]
    transcript = []
    budget = Budget(max_turns=turns, max_tokens=4000, max_seconds=120)
```

```

while not budget.exhausted():
    speaker = agents[len(transcript) % 2]

```

The setup, then the loop. The speaker is chosen by parity: an even number of messages so far means it is A's turn, odd means B's.

```

messages = [
    {"role": "system",
     "content": f"{speaker.system_prompt}\nDebate topic: {TOPIC}"
    },
    {"role": "user",
     "content": f"Conversation so far:\n{render(transcript)}\n\n"
               f"Your turn, {speaker.name}. Reply with one"
               f"short message."
    },
]

```

This is the naive part. The whole conversation is pasted into a single user message as text. The model has no structural way of knowing which lines were its own and which were the other agent's; it has to work that out from the names. The message also grows on every turn, which is the problem Week 3 deals with.

```

reply = client.chat(speaker.model, messages, temperature=speaker.
temperature)
transcript.append((speaker.name, reply.text))
budget.record(turns=1, tokens=reply.tokens)
print(f"{speaker.name}: {reply.text}\n")

print(f"[stopped: {budget.stop_reason} ...]")

```

Call the model, keep the reply, charge the budget, print it. When the loop ends, say why. The `__main__` block at the bottom reads `-mock` and `-turns` from the command line and calls `main`.

## 6 engine.py: the Week 2 target

### 6.1 Entry

```

@dataclass
class Entry:
    speaker: str
    content: str
    prompt_tokens: int
    completion_tokens: int
    seconds: float
    turn_index: int

```

One message in the transcript. Compared with the `(name, text)` tuple of Week 1 it also carries the cost of producing the message and its position, which is what you will need when you start measuring things in Week 4.

### 6.2 view\_for: to be written

The function currently raises `NotImplementedError`. Its docstring specifies what it should return: a list of message dictionaries built as follows. The first is the agent's system prompt with role `system`. Then, for each entry in the transcript, one message whose role is `assistant` if

that entry was spoken by this agent and `user` if it was spoken by the other. If the transcript is empty there would otherwise be no user message at all, so in that case a starter such as "You speak first." is added with role `user`.

The effect is that each agent sees its own past lines as things it said and the other agent's lines as things said to it, which is the way chat models are trained to read a conversation. The two agents' views of the same transcript are mirror images of each other. This is the whole point of Week 2 and the reason `ping_pong.py` is retired.

### 6.3 DialogueEngine

```
def __init__(self, agents, client, budget, goal_reached=None,
             manage_context=None):
    self.agents = agents
    self.client = client
    self.budget = budget
    self.transcript = []
    self.goal_reached = goal_reached or (lambda t: False)
    self.manage_context = manage_context or (lambda messages: messages)

def next_speaker(self):
    return self.agents[len(self.transcript) % len(self.agents)]
```

The constructor stores what it is given. The two optional arguments are hooks, and if you do not supply them they default to functions that do nothing: `goal_reached` always answers "no" and `manage_context` returns the messages untouched. Week 3 replaces the second one with truncation or summarisation. `next_speaker` is the parity trick from before, generalised so that any number of agents take turns in list order.

### 6.4 run: to be written

The docstring lays the loop out almost line for line. Each iteration: check the budget; pick the speaker; build its view with `view_for` and pass it through `manage_context`; call the client; wrap the reply in an `Entry`, using the current transcript length as `turn_index`; record the cost; and if `goal_reached` says the scenario is finished, tell the budget to stop with that reason. When the loop exits, return the transcript.

### 6.5 save: done

```
def save(self, path, meta=None):
    import os
    os.makedirs(os.path.dirname(path) or ".", exist_ok=True)
    record = {
        "meta": meta or {},
        "agents": [{"name": a.name, "model": a.model, "temperature": a.
temperature}
                    for a in self.agents],
        "budget": self.budget.summary(),
        "totals": { ... sums over the transcript ... },
        "messages": [asdict(e) for e in self.transcript],
    }
    with open(path, "w") as f:
        json.dump(record, f, indent=2)
    return path
```

Makes sure the output directory exists (the `or "."` handles a bare filename with no directory part), then assembles everything about the run into one dictionary: the metadata the caller passed in, a description of each agent, the budget's summary, some totals, and every entry converted to a dictionary by `asdict`. `json.dump` with `indent=2` writes it out in a form you can read by eye.

## 7 `run.py`: the entry point

```
def smoke(mock):
    client = make_client(mock=mock)
    messages = [
        {"role": "system", "content": "You are terse."},
        {"role": "user", "content": "Say hello in exactly three words."},
    ]
    r = client.chat("llama3.2:3b", messages, temperature=0)
    print("Model replied:", r.text)
```

A single call to prove the setup works. Temperature zero makes the reply deterministic, so if it works once it will keep working.

```
def run_config(path, mock):
    import yaml
    from agents import Agent
    from budget import Budget
    from engine import DialogueEngine

    with open(path) as f:
        cfg = yaml.safe_load(f)
        agents = [Agent(**a) for a in cfg["agents"]]
        budget = Budget(**cfg.get("budget", {}))
        client = make_client(mock=mock)

    engine = DialogueEngine(agents, client, budget)
    transcript = engine.run()
    for e in transcript:
        print(f"{e.speaker}: {e.content}\n")
    out = cfg.get("output", "transcripts/run.json")
    engine.save(out, meta={"topic": cfg.get("topic"), "config": path})
```

The imports are inside the function on purpose: the smoke test should still run even if `engine.py` is unfinished or PyYAML is missing. The config is loaded with `yaml.safe_load`, and then `Agent(**a)` unpacks each agent's mapping as keyword arguments. This only works because the keys in the YAML file match the field names of the dataclass exactly, which is a constraint to remember when you write your own configs. The same trick builds the `Budget`. After that it is run, print, save.

The block at the bottom decides which of the two to call: a full run if `-config` was given, the smoke test otherwise.

## 8 `debate.yaml`: the config

YAML is a plain-text format for configuration. Indentation shows structure, `key: value` makes a mapping, and lines beginning with `-` make a list. This file has four top-level keys.

```
|topic: "Cities should ban private cars from their centres."
```

A string. The engine never reads it; `run.py` copies it into the saved transcript's meta so that you can tell later what a run was about.

```
budget:
  max_turns: 8
  max_tokens: 4000
  max_seconds: 120
```

A mapping of three integers. In `run.py` this becomes `Budget(**cfg["budget"])`. The `**` operator unpacks a dictionary into keyword arguments, so the call is exactly

```
Budget(max_turns=8, max_tokens=4000, max_seconds=120)
```

This is why the key names must match the parameter names of `Budget.__init__`: a typo such as `max_turn` would produce a `TypeError` rather than a silent default.

```
agents:
  - name: Pro
    system_prompt: >
      Debate topic: Cities should ban private cars from their centres.
      You argue IN FAVOUR. Make one concrete point per turn.
      Be brief: at most 2 sentences. Address the other speaker's last
      point.
    model: "llama3.2:3b"
    temperature: 0.7
  - name: Con
  ...
```

A list of two mappings, one per agent. Each has the same four keys as the `Agent` dataclass, so `Agent(**a)` works for each of them. The `>` after `system_prompt:` is YAML's folded block: the indented lines beneath it are joined into a single string, with a space where each line break was, so the model receives one long paragraph rather than three separate lines. (The alternative marker, `|`, would keep the newlines.)

Order matters here. The first agent in the list speaks first, because `next_speaker` walks the list in order.

```
|output: "transcripts/debate.json"
```

Where `save` writes. The directory is created if it does not exist.

The comment at the top of the file says to copy it per experiment and change one thing at a time. That is the Week 3 method: if you change temperature and turn limit together and the results differ, you cannot tell which one mattered.

## 9 test\_engine.py: the self-check

This is an ordinary script rather than a `pytest` suite. It runs from top to bottom, and each `assert` either passes silently or raises `AssertionError` carrying the message written after the comma.

### 9.1 Setup

```
A = Agent("Alice", "You are Alice.")
B = Agent("Bob", "You are Bob.")
```

Two throwaway agents with the default model and temperature.

## 9.2 Checking view\_for

```
transcript = [
    Entry("Alice", "Hi Bob.", 10, 5, 0.1, 0),
    Entry("Bob", "Hey Alice.", 12, 6, 0.1, 1),
    Entry("Alice", "How's it going?", 14, 7, 0.1, 2),
]
alice_view = view_for(A, transcript)
bob_view = view_for(B, transcript)
```

A three-message transcript built by hand. The positional arguments follow the field order of `Entry`: speaker, content, prompt tokens, completion tokens, seconds, turn index. The numbers are arbitrary. The same transcript is then rendered from each side.

```
assert alice_view[0] == {"role": "system", "content": "You are Alice."}
assert alice_view[1]["role"] == "assistant"
assert alice_view[2]["role"] == "user"
assert alice_view[3]["role"] == "assistant"
```

Alice's list should be her system prompt, then her line (assistant), Bob's line (user), her line again (assistant). Because index 0 is the system message, transcript entry  $i$  sits at index  $i + 1$ .

```
assert bob_view[0] == {"role": "system", "content": "You are Bob."}
assert bob_view[1]["role"] == "user"
assert bob_view[2]["role"] == "assistant"
```

Bob's view is the mirror image: the same transcript with the roles swapped.

```
empty_view = view_for(A, [])
assert len(empty_view) >= 2
assert empty_view[0]["role"] == "system"
assert empty_view[1]["role"] == "user"
```

With no history the function must still return a system message followed by a user message; otherwise the model would be asked to reply to nothing. This is the "You speak first" case from the docstring.

## 9.3 Checking run

```
client = make_client(mock=True)
budget = Budget(max_turns=4, max_tokens=4000, max_seconds=60)
engine = DialogueEngine([A, B], client, budget)
result = engine.run()

assert len(result) == 4
assert result[0].speaker == "Alice"
assert result[1].speaker == "Bob"
assert result[2].speaker == "Alice"
assert budget.stop_reason == "max_turns"
```

A real engine run against the fake model, capped at four turns. Exactly four entries must come back, alternating and starting with Alice, and the run must have ended because of the turn cap. The token and time limits are set high enough that they cannot be the reason.

```
for i, e in enumerate(result):
    assert e.turn_index == i
    assert e.prompt_tokens > 0
    assert e.completion_tokens > 0
```

`enumerate` yields the index alongside each entry. Every entry's `turn_index` must equal its position, and both token counts must be positive. The second check catches the common mistake of forgetting to copy the numbers out of `reply` when building the `Entry`.

## 9.4 Checking save

```
path = engine.save("transcripts/_test.json", meta={"test": True})
assert os.path.exists(path)
with open(path) as f:
    data = json.load(f)
assert len(data["messages"]) == 4
assert data["budget"]["stop_reason"] == "max_turns"
os.remove(path)
```

Write the transcript, confirm the file exists, read it back with `json.load`, check that it contains what was written, and delete it so the test leaves nothing behind. If the script ends with `All checks passed.`, the Week 2 work is done.

## 10 Smaller pieces not covered above

### 10.1 README.md and requirements.txt

The README is not code, but it settles two practical points. The model tag must be exactly `llama3.2:3b`; pulling `llama3.2` gives you a different, larger model and the 404 branch in `OllamaClient` will fire. And `-mock` works with every command, so the entire engine can be built and debugged offline.

`requirements.txt` has two lines: `requests`, used by `OllamaClient` for HTTP, and `PyYAML`, used by `run_config`. Everything else the project imports (`dataclasses`, `json`, `time`, `os`, `argparse`) is in the standard library.

### 10.2 The \_\_main\_\_ blocks

Both `ping_pong.py` and `run.py` end with

```
if __name__ == "__main__":
    p = argparse.ArgumentParser()
    p.add_argument("--mock", action="store_true", help="...")
    p.add_argument("--turns", type=int, default=8, help="...")
    args = p.parse_args()
    main(mock=args.mock, turns=args.turns)
```

When Python runs a file directly, it sets the variable `__name__` to the string `"__main__"`. When the file is imported by another module instead, that variable holds the module's own name. So the code under this guard runs when you type `python run.py` but not when `test_engine.py` imports from it. Inside, `ArgumentParser` declares the command-line options. `action="store_true"` makes `-mock` a simple flag that is either present or absent; `type=int` makes `-turns` parse a number; `parse_args` reads the actual command line and returns an object whose attributes (`.mock`, `.turns` and so on) hold the values.

### 10.3 MockClient.DEFAULT\_REPLIES

```
DEFAULT_REPLIES = [
    "I think we should weigh the costs before anything else.",
    "Fair, but the long-term benefits clearly outweigh those costs.",
```

```

    "Only if we ignore the people who can't adapt quickly.",
    "We can support them with a transition plan; that's solvable.",
    "Then let's agree the plan matters as much as the decision.",
    "Agreed. I think that's our common ground.",
]

```

Six lines written to sound like a debate that converges on agreement. That is deliberate. When you write a `goal_reached` judge in Week 3, the last two replies give it something to detect. Because the counter wraps with %, a run longer than six turns starts repeating from the top.

## 10.4 Budget.summary

```

def summary(self):
    return {
        "stop_reason": self.stop_reason,
        "turns": self.turns,
        "tokens": self.tokens,
        "seconds": round(self.elapsed, 3),
        "limits": {
            "max_turns": self.max_turns,
            "max_tokens": self.max_tokens,
            "max_seconds": self.max_seconds,
        },
    }

```

A nested dictionary: the stop reason, the three counters (with elapsed time rounded to three decimals), and a `limits` sub-dictionary holding the three caps. It exists so a saved transcript records not only what happened but what the rules were, which you need when comparing experiments in Week 4.

## 10.5 The `totals` block in `save`

```

"totals": {
    "turns": len(self.transcript),
    "prompt_tokens": sum(e.prompt_tokens for e in self.transcript),
    "completion_tokens": sum(e.completion_tokens for e in self.transcript),
    "seconds": round(sum(e.seconds for e in self.transcript), 3),
},
"messages": [asdict(e) for e in self.transcript],

```

The `sum(... for e in ...)` constructions are generator expressions: for each entry take one field, and add them up. `prompt_tokens` grows quickly over a run because every turn re-sends the whole history; shrinking that number is what Week 3's context management is for.

`asdict(e)` converts a dataclass instance into an ordinary dictionary, field by field. This is needed because `json.dump` can serialise dictionaries and lists but not arbitrary objects.

## 10.6 The `manage_context` hook

The hook receives the message list that `view_for` produced and must return a message list. The default is `lambda messages: messages`, which changes nothing. In Week 3 you replace it with a function that drops or summarises the middle of the conversation while

keeping the system message at index 0 and the most recent turns at the end, so that the prompt stays inside the model's context window no matter how long the dialogue runs.

## **The one rule**

No `while` loop that calls a model exists without a `Budget` controlling it. When in doubt, start with `Budget (max_turns=2)` in mock mode. Two turns cannot hurt.